

Hybridation d'une métaheuristique de groupement et de l'apprentissage par renforcement pour le problème du Bin Packing unidimensionnel

Mostefai Mounir Sofiane, Tati Youcef, Badaoui Ikram, Bousdjira Nadine, Hasnaoui Sarah, Zitouni Rania

Équipe Square Two – Module Optimisation

École Nationale Supérieure d'Informatique (ESI)

Alger, Algérie

Emails : mm_mostefai@esi.dz, my_tati@esi.dz, mi_badaoui@esi.dz, mn_bousdjira@esi.dz, ms_hasnaoui@esi.dz, mr_zitouni@esi.dz

Résumé—Le *Bin Packing Problem* unidimensionnel (1D-BPP) consiste à affecter un ensemble d'objets de tailles connues à un nombre minimal de conteneurs identiques de capacité fixe, sans dépasser la capacité de chaque conteneur. Ce problème est NP-difficile et devient rapidement coûteux pour les méthodes exactes lorsque la taille des instances augmente. Dans ce travail, nous retenons comme axe central une hybridation entre la métaheuristique *Hybrid Grouping Genetic Algorithm* (HGGA) et l'apprentissage par renforcement. L'approche proposée, notée RL-HGGA, conserve les opérateurs classiques de HGGA mais remplace leur sélection probabiliste fixe par une politique adaptative apprise par Q-learning. L'agent observe l'état courant de la recherche, choisit une macro-action de recherche, puis met à jour sa politique à partir d'une récompense fondée sur l'amélioration de la solution et le coût temporel. Les expérimentations sont menées sur des benchmarks standards du 1D-BPP (Falkenauer, Scholl et Hard28) et comparent FFD, HGGA et RL-HGGA en termes de nombre de bins, d'écart à la borne inférieure et de temps d'exécution. Les résultats montrent que HGGA reste la meilleure référence en qualité moyenne, alors que RL-HGGA fournit un compromis qualité-temps intéressant en réduisant fortement les temps de calcul tout en conservant une qualité proche de celle de HGGA. L'étude d'ablation montre que le composant ML améliore surtout l'efficacité temporelle de la recherche, tout en conservant une qualité proche de celle de HGGA.

Index Terms—Bin Packing Problem, optimisation combinatoire, métaheuristic, Grouping Genetic Algorithm, HGGA, apprentissage par renforcement, Q-learning, hybridation.

I. INTRODUCTION

A. Définition du problème

Le *Bin Packing Problem* unidimensionnel est un problème de regroupement et d'affectation. On dispose d'un ensemble d'objets $I = \{1, \dots, n\}$, chaque objet i ayant une taille w_i , et d'un ensemble de conteneurs identiques de capacité C . Il faut affecter chaque objet à un conteneur de sorte que la capacité ne soit jamais dépassée, tout en minimisant le nombre total de conteneurs utilisés. Ce problème intervient naturellement en logistique, découpe industrielle, allocation mémoire, stockage de fichiers et chargement de ressources. Sa difficulté théorique est bien connue : le 1D-BPP est NP-difficile, ce qui explique

le recours massif à des méthodes approchées lorsque la taille des instances augmente [1], [2].

B. État de l'art des métaheuristic

La littérature de résolution du 1D-BPP peut être organisée en trois grandes familles. La première regroupe les heuristiques constructives, telles que *First Fit*, *Best Fit*, *First Fit Decreasing* (FFD) ou *Best Fit Decreasing*. Ces méthodes construisent une solution en parcourant les objets selon un ordre donné et en les plaçant dans un bin existant ou dans un nouveau bin. Elles sont appréciées pour leur simplicité et leur rapidité, mais elles restent limitées par leur caractère glouton : une mauvaise décision prise au début de la construction n'est généralement pas remise en cause par la suite [2].

La deuxième famille correspond aux méthodes exactes et quasi-exactes, comme la programmation linéaire en nombres entiers, le *branch-and-bound* ou les schémas de *branch-and-price*. Ces approches sont importantes car elles peuvent prouver l'optimalité ou fournir des bornes de référence. Cependant, leur coût de calcul augmente rapidement avec la taille et la difficulté des instances. Dans un contexte expérimental où l'on veut traiter plusieurs familles de benchmarks, les méthodes approchées restent donc nécessaires.

La troisième famille, centrale dans ce travail, est celle des métaheuristic. Les métaheuristic sont des méthodes générales de recherche capables d'explorer un espace de solutions très grand sans l'énumérer complètement. Elles peuvent être basées sur une population, comme les algorithmes génétiques, les colonies de fourmis ou les algorithmes à essaim, ou sur une solution unique, comme la recherche tabou, le recuit simulé ou certaines recherches locales avancées. Leur intérêt est de combiner **exploration**, pour visiter différentes régions de l'espace de recherche, et **exploitation**, pour améliorer localement les solutions prometteuses.

Dans une étude comparative consacrée au 1D-BPP, Munien *et al.* comparent plusieurs approches métaheuristic inspirées de la nature, notamment des variantes de *Firefly Algorithm*, *Artificial Bee Colony*, *Cuckoo Search*, *Cuckoo Search* hybridé avec un algorithme génétique, ainsi qu'un

algorithme génétique classique [10]. Leur étude montre que les métaheuristiques peuvent obtenir des solutions de bonne qualité sur des instances de difficulté variable, mais aussi que le temps de calcul et la qualité dépendent fortement de l'algorithme choisi, de l'heuristique interne utilisée et de la structure des instances. Cette observation confirme que le 1D-BPP ne se résume pas à choisir une métaheuristique performante en moyenne : il faut aussi contrôler son comportement selon l'état de la recherche.

Dans le cas spécifique du BPP, une avancée importante est le *Grouping Genetic Algorithm* (GGA), puis sa version hybride HGGA proposée par Falkenauer [3]. Contrairement à un algorithme génétique classique qui représenterait une solution par une permutation d'objets, le GGA représente directement une solution comme un ensemble de groupes, chaque groupe correspondant à un bin. Cette représentation est plus naturelle pour le BPP, car l'information essentielle n'est pas seulement l'ordre des objets, mais la manière dont ils sont regroupés dans les bins.

Les métaheuristiques classiques présentent cependant plusieurs limites. D'abord, elles dépendent fortement du réglage des paramètres : taille de population, taux de mutation, probabilité de recherche locale, intensité de diversification, critère d'arrêt, etc. Ensuite, les probabilités d'application des opérateurs sont souvent fixes, alors qu'un même opérateur n'est pas forcément utile à toutes les étapes de la recherche. Par exemple, une mutation forte peut être utile en cas de stagnation, mais inutile ou coûteuse lorsque la population progresse déjà. Enfin, les métaheuristiques peuvent souffrir de convergence prématurée, de perte de diversité ou d'un coût de calcul élevé. Ces limites motivent l'ajout d'un mécanisme adaptatif capable de piloter la recherche en fonction de son état courant.

C. Motivation scientifique de l'hybridation avec le ML

L'idée d'hybrider une métaheuristique avec du *Machine Learning* (ML) repose sur une observation simple : de nombreuses décisions internes d'un algorithme d'optimisation sont aujourd'hui encore pilotées par des règles fixes ou par des probabilités statiques. Or, au cours d'une exécution, l'état de la recherche évolue : qualité actuelle, diversité de la population, stagnation, proximité d'une borne inférieure ou coût des opérateurs. Il est donc naturel de chercher un mécanisme capable d'adapter dynamiquement les décisions de l'algorithme à cet état courant.

Plusieurs modes d'hybridation sont possibles : apprentissage d'un réglage de paramètres, sélection d'opérateurs, choix d'un voisinage, apprentissage d'une politique de diversification, ou encore prédiction d'un profil de résolution à partir des caractéristiques d'une instance. Dans notre cas, nous retenons la sélection d'opérateurs par apprentissage par renforcement, car cette décision se répète à chaque génération et influence directement le compromis exploration-exploitation. Le Q-learning est particulièrement adapté à ce type de problème séquentiel : l'agent observe un état, choisit une action, reçoit

une récompense, puis ajuste progressivement sa politique [5], [6].

D. État de l'art de l'hybridation ML-métaheuristiques

L'hybridation entre ML et métaheuristiques s'inscrit dans un cadre plus large, celui des *hyper-heuristics* et des méthodes d'optimisation assistées par apprentissage. Une première ligne de travaux concerne les *selection hyper-heuristics*, où un mécanisme de haut niveau choisit dynamiquement l'heuristique ou l'opérateur à appliquer parmi un portefeuille donné. Cette idée a montré son intérêt sur de nombreux problèmes combinatoires, car elle permet de remplacer des stratégies rigides par des politiques adaptatives [7].

Une deuxième ligne de travaux concerne l'usage de l'apprentissage par renforcement pour piloter des décisions locales à l'intérieur d'un algorithme d'optimisation. L'agent apprend alors, à partir d'interactions répétées, quelles actions sont les plus rentables selon la phase de recherche. Ce principe a été exploré dans plusieurs problèmes NP-difficiles, notamment en ordonnancement, en routage et en optimisation combinatoire plus générale, où il sert à sélectionner des voisins, des règles de priorité ou des opérateurs [8], [9].

Dans le cas spécifique du 1D-BPP, la littérature reste plus abondante sur les heuristiques et métaheuristiques classiques que sur les approches explicitement guidées par apprentissage. Cela renforce l'intérêt d'une étude comme la nôtre : au lieu de modifier la structure de HGGA, nous cherchons à conserver ses opérateurs efficaces et à ne rendre adaptatif que leur mécanisme de sélection. Cette stratégie permet de mieux isoler l'apport du composant ML, tout en restant fidèle à une métaheuristique de référence reconnue pour le BPP [3].

E. Contribution et originalité

L'objectif de ce travail n'est pas de proposer une nouvelle métaheuristique entièrement différente, mais de concevoir une hybridation ciblée, cohérente et analysable entre HGGA et l'apprentissage par renforcement. Les contributions principales sont les suivantes :

- proposer une version **RL-HGGA** dans laquelle le composant ML est inséré exactement au point critique de décision de l'algorithme, à savoir la sélection de l'opérateur à appliquer à chaque génération ;
- conserver l'ossature de HGGA (encodage par groupes, croisement BPCX, mutation, recherche locale, sélection et remplacement), afin que l'effet du composant intelligent puisse être isolé et interprété ;
- définir un **état de recherche** composé de descripteurs informatifs de la population et de la progression de l'algorithme ;
- définir un **espace d'actions** sous forme de macro-actions associant les opérateurs de HGGA suivant plusieurs intensités de recherche ;
- justifier le **positionnement du composant ML** : l'agent est placé à l'intérieur de la boucle évolutionnaire, là où la décision doit être répétée, contextuelle et dépendante du comportement courant de la population ;

— mener une **étude d'ablation** en comparant HGGA sans ML avec RL-HGGA, afin d'évaluer l'apport réel du composant Q-learning.

L'originalité du travail tient donc moins à l'invention de nouveaux opérateurs qu'à la manière dont ils sont orchestrés : au lieu d'un contrôle fixe, RL-HGGA apprend une politique d'application contextualisée, ce qui permet d'analyser explicitement l'apport d'un agent de Q-learning sur une métaheuristique de groupement dédiée au BPP.

F. Organisation du papier

La section II formalise le 1D-BPP. La section III présente l'approche RL-HGGA proposée, son architecture générale, son codage et le composant Q-learning. La section IV décrit le protocole expérimental, les jeux de données, l'environnement matériel et logiciel, ainsi que les réglages retenus. La section V analyse les résultats expérimentaux et l'effet réel de l'hybridation. Enfin, la section VI conclut le travail et propose des perspectives.

II. FORMULATION MATHÉMATIQUE DU PROBLÈME

A. Définition formelle

Soit un ensemble d'objets $I = \{1, 2, \dots, n\}$, où chaque objet i possède une taille positive w_i . Les objets doivent être placés dans des bins identiques de capacité C . On cherche à minimiser le nombre de bins utilisés. Comme un objet peut toujours être placé seul dans un bin, un nombre maximal trivial de bins est $U = n$.

B. Variables de décision

On définit deux types de variables binaires :

$$x_{ij} = \begin{cases} 1, & \text{si l'objet } i \text{ est placé dans le bin } j, \\ 0, & \text{sinon,} \end{cases} \quad (1)$$

avec $i \in \{1, \dots, n\}$ et $j \in \{1, \dots, U\}$.

On définit également :

$$y_j = \begin{cases} 1, & \text{si le bin } j \text{ est utilisé,} \\ 0, & \text{sinon.} \end{cases} \quad (2)$$

C. Fonction objectif

La fonction objectif consiste à minimiser le nombre total de bins utilisés :

$$\min Z = \sum_{j=1}^U y_j. \quad (3)$$

D. Contraintes

Chaque objet doit être affecté à exactement un bin :

$$\sum_{j=1}^U x_{ij} = 1, \quad \forall i \in \{1, \dots, n\}. \quad (4)$$

La capacité de chaque bin ne doit pas être dépassée :

$$\sum_{i=1}^n w_i x_{ij} \leq C y_j, \quad \forall j \in \{1, \dots, U\}. \quad (5)$$

Les variables sont binaires :

$$x_{ij} \in \{0, 1\}, \quad y_j \in \{0, 1\}. \quad (6)$$

Une contrainte optionnelle de rupture de symétrie peut être ajoutée afin d'éviter les solutions équivalentes obtenues par permutation des bins :

$$y_j \geq y_{j+1}, \quad \forall j \in \{1, \dots, U-1\}. \quad (7)$$

E. Borne inférieure utilisée dans l'évaluation

Une borne inférieure naturelle est donnée par :

$$LB = \left\lceil \frac{\sum_{i=1}^n w_i}{C} \right\rceil. \quad (8)$$

Dans les expérimentations, cette borne sert à normaliser l'évaluation et à calculer un écart relatif entre méthodes :

$$Gap(\%) = \frac{Bins_{methode} - LB}{LB} \times 100. \quad (9)$$

III. DESCRIPTION DE L'APPROCHE HYBRIDE PROPOSÉE

A. Architecture globale

La solution proposée repose sur une métaheuristique de base, HGGA, ensuite pilotée par apprentissage par renforcement. Une instance du BPP est d'abord transformée en une population initiale de solutions faisables à l'aide d'une version bruitée de FFD. Cette population est ensuite améliorée par une boucle évolutionnaire de type HGGA : sélection des parents, application d'opérateurs de variation et d'amélioration, évaluation des solutions, puis remplacement des individus les moins performants.

La différence essentielle entre HGGA classique et RL-HGGA se situe au niveau du choix des opérateurs. Dans HGGA, le choix du croisement, de la mutation ou de la recherche locale est contrôlé par des probabilités fixées à l'avance. Dans RL-HGGA, ce choix est confié à un agent Q-learning. À chaque génération, l'agent observe l'état de la recherche, sélectionne une macro-action, puis reçoit une récompense selon l'effet obtenu sur la qualité de la population et le coût temporel.

La Fig. 1 est réservée au schéma final de l'architecture. Elle doit montrer clairement la boucle HGGA principale et le point d'insertion du composant ML : l'agent Q-learning intervient uniquement pour piloter la sélection des opérateurs, sans remplacer les opérateurs de HGGA.

B. Principe détaillé de HGGA

HGGA signifie *Hybrid Grouping Genetic Algorithm*. Il s'agit d'une adaptation des algorithmes génétiques aux problèmes de groupement. Dans un algorithme génétique classique, une population de solutions évolue grâce à des opérateurs comme la sélection, le croisement et la mutation. Ce schéma général reste présent dans HGGA, mais il est modifié pour mieux correspondre à la structure du Bin Packing.

La particularité de HGGA est que l'unité importante n'est pas seulement l'objet, mais le **groupe d'objets**. Dans le BPP, un groupe correspond à un bin. Ainsi, l'objectif de l'algorithme est de faire évoluer des regroupements d'objets et

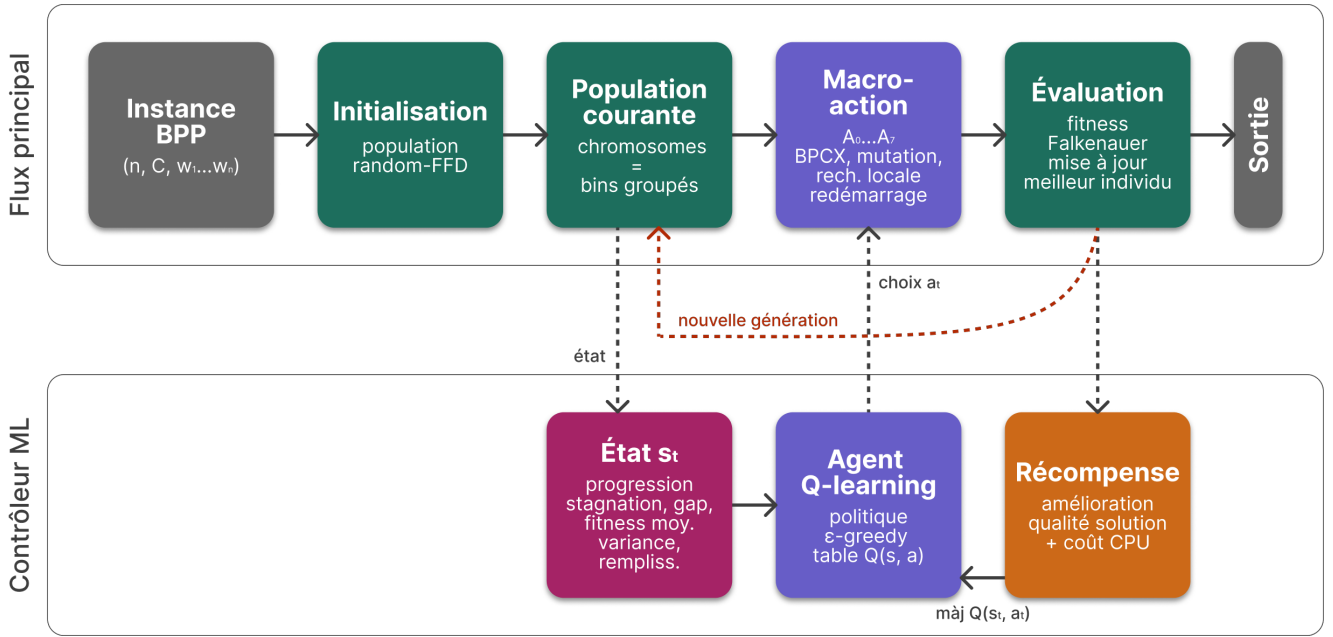


FIGURE 1. Architecture détaillée de RL-HGGA et positionnement du composant ML dans la boucle évolutive.

de préserver les bins bien construits au cours des générations. Cette idée est essentielle : si un bin est presque plein sans dépasser la capacité, il représente une bonne structure qu'il faut essayer de conserver lors du croisement.

Le fonctionnement général de HGGA peut être résumé en quatre étapes. Premièrement, une population initiale de solutions faisables est générée. Deuxièmement, des parents sont sélectionnés, généralement par tournoi, afin de favoriser les individus de bonne qualité sans supprimer totalement la diversité. Troisièmement, des opérateurs spécialisés produisent de nouveaux individus : le croisement BPCX transmet des groupes entiers d'un parent à l'autre, la mutation dissout certains bins puis réinsère leurs objets, et la recherche locale améliore les regroupements par des déplacements ou échanges d'objets. Quatrièmement, un remplacement de type $(\mu + \lambda)$ conserve les meilleurs individus parmi les parents et les descendants.

HGGA est donc une métaheuristique hybride pour deux raisons. D'une part, elle combine une logique évolutionnaire globale, qui explore plusieurs régions de l'espace de recherche à travers une population. D'autre part, elle intègre des mécanismes d'amélioration locale et de réparation, qui exploitent les bonnes solutions déjà trouvées. Cette combinaison permet d'obtenir de meilleures solutions qu'une heuristique constructive simple comme FFD, mais elle augmente aussi le temps de calcul. C'est précisément cette limite qui motive l'ajout d'un composant Q-learning dans RL-HGGA.

C. Codage et représentation des solutions

Le codage implémenté dans les notebooks suit fidèlement la logique du *Grouping Genetic Algorithm*. Une solution n'est pas représentée par une simple permutation d'objets, mais par

un **chromosome** composé d'une liste ordonnée de **groupes**. Chaque groupe correspond à un bin.

Dans l'implémentation, un Group contient une liste de couples $(item_id, weight)$ ainsi qu'un attribut *fill* représentant la charge totale du bin. Cette structure permet de tester immédiatement la faisabilité d'une insertion via la condition $fill + w \leq C$. Un Chromosome est ensuite défini comme une liste de groupes, c'est-à-dire un empaquetage complet.

Cette représentation offre plusieurs avantages pratiques et algorithmiques.

- Elle est **naturelle** pour le BPP, puisque le problème consiste précisément à partitionner les objets en bins.
- Elle facilite la **préservation de bons groupes** pendant le croisement, contrairement à un codage par permutation où l'information de groupement est implicite.
- Elle rend la **réparation et l'amélioration locale** plus simples, car on peut déplacer, échanger ou réinsérer directement des objets entre groupes.

La qualité d'un chromosome est évaluée par la fitness de Falkenauer :

$$fitness(X) = \frac{1}{|X|} \sum_{g \in X} \left(\frac{fill(g)}{C} \right)^k, \quad (10)$$

avec $k = 2$ dans l'implémentation. Cette fonction favorise les bins bien remplis et pénalise les remplissages dispersés, ce qui est cohérent avec l'objectif de minimisation du nombre de bins [3].

D. Initialisation de la population

La population initiale n'est pas générée aléatoirement de manière uniforme. Les notebooks utilisent une variante

random-FFD : les objets sont d'abord mélangés, puis partiellement triés par blocs avant application de *First Fit*. Cette idée introduit de la diversité tout en conservant une bonne qualité initiale. En pratique, chaque individu initial est donc une solution faisable, relativement compétitive, mais différente des autres. Cette étape est importante, car elle fournit à HGGA une population de départ à la fois variée et déjà structurée.

E. Opérateurs de HGGA conservés dans RL-HGGA

RL-HGGA conserve exactement les opérateurs de HGGA ; seul leur mode de sélection change.

Croisement BPCX. Le *Bin Packing Crossover* (BPCX) sélectionne un segment de groupes dans un parent, injecte des groupes du second parent, supprime les duplications d'objets, puis réinsère les objets orphelins par un mécanisme de type FFD. Le but est de préserver des bins de bonne qualité au lieu de casser systématiquement les regroupements.

Mutation légère et lourde. Les mutations sont implémentées par dissolution partielle de bins, suivie d'une réinsertion gloutonne. La mutation légère dissout une faible fraction de bins peu remplis, tandis que la mutation lourde dissout une fraction plus importante. Dans le code, cette logique est réalisée par `dissolve_and_repack`, avec un paramètre de fraction plus ou moins élevé.

Recherche locale. La recherche locale utilisée est de style Martello–Toth. Les bins les moins remplis sont examinés en priorité, et l'algorithme tente des mouvements de dominance de type remplacement de deux objets par un, ou l'inverse, afin de supprimer des bins peu efficaces et de densifier le regroupement [4].

Sélection et remplacement. Les parents sont choisis par tournoi, et le remplacement suit une stratégie $(\mu + \lambda)$: parents et descendants sont réunis, triés par fitness, puis seuls les meilleurs individus sont conservés. Cette stratégie assure une pression de sélection stable tout en évitant de perdre immédiatement de bonnes solutions.

F. Composant Q-learning : états, actions et récompense

L'agent de Q-learning est le coeur du module intelligent. À chaque génération, il reçoit un état discrétisé de la recherche. Cet état comporte huit caractéristiques continues normalisées, ensuite discrétisées pour alimenter une table Q tabulaire :

- progression de la génération ;
- niveau de stagnation ;
- gap de la meilleure solution par rapport à la borne inférieure ;
- fitness moyenne de la population ;
- variance de fitness ;
- fraction de bins presque pleins ;
- fraction de bins peu remplis ;
- pente récente d'amélioration.

L'espace d'actions est composé de huit **macro-actions** :

- A0 : BPCX + mutation légère ;
- A1 : BPCX + mutation légère + recherche locale ;
- A2 : BPCX + mutation lourde ;
- A3 : BPCX + mutation lourde + recherche locale ;

- A4 : BPCX + recherche locale ;
- A5 : recherche locale seule sur le meilleur individu ;
- A6 : redémarrage des pires individus ;
- A7 : mutation seule.

Le choix de ces actions est important : l'agent n'apprend pas un opérateur élémentaire abstrait, mais choisit directement une **recette de recherche** correspondant à un style d'évolution donné. Cela rend l'apprentissage plus interprétable.

Pour la variante principale RL-HGGA, la récompense combine l'amélioration de la meilleure fitness, l'amélioration de la fitness moyenne et une pénalité liée au coût CPU de l'action. La règle de mise à jour est celle du Q-learning classique :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]. \quad (11)$$

La conception de cette récompense est centrale, car elle détermine le compromis entre amélioration de la solution et coût temporel des opérateurs.

Algorithm 1 Schéma général de RL-HGGA

- 1: Générer une population initiale avec *random-FFD*
 - 2: Initialiser l'agent Q-learning
 - 3: **for** chaque génération **do**
 - 4: Extraire l'état courant s_t
 - 5: Choisir une action a_t avec une politique ϵ -greedy
 - 6: Appliquer la macro-action HGGA correspondant à a_t
 - 7: Évaluer la nouvelle population et mettre à jour le meilleur individu
 - 8: Calculer la récompense r_t
 - 9: Observer le nouvel état s_{t+1}
 - 10: Mettre à jour $Q(s_t, a_t)$
 - 11: **end for**
 - 12: Retourner le meilleur chromosome obtenu
-

IV. EXPÉRIMENTATIONS, RÉSULTATS ET ANALYSES

A. Protocole expérimental

Le protocole expérimental a pour objectif d'évaluer l'apport de l'apprentissage par renforcement dans la métaheuristique HGGA pour le problème du Bin Packing unidimensionnel. Trois méthodes principales sont comparées : FFD, utilisée comme baseline heuristique rapide ; HGGA, utilisée comme baseline métaheuristique sans composant intelligent ; et RL-HGGA, qui constitue l'approche hybride proposée.

Le scénario de test est organisé en deux phases. Dans une première phase, l'agent Q-learning de RL-HGGA est entraîné sur un ensemble diversifié d'instances afin d'apprendre une politique de sélection des opérateurs. Pendant cette phase, l'agent choisit les actions à l'aide d'une stratégie ϵ -greedy, ce qui permet d'équilibrer exploration et exploitation. Dans une deuxième phase, les méthodes sont évaluées sur les familles de benchmarks retenues. Pour RL-HGGA, l'évaluation utilise la politique gloutonne apprise, c'est-à-dire que l'agent sélectionne à chaque génération l'action ayant la meilleure valeur $Q(s, a)$ pour l'état courant.

Toutes les méthodes sont exécutées sur les mêmes familles d’instances et comparées à l’aide des mêmes indicateurs : le nombre moyen de bins utilisés, le gap moyen par rapport à la borne inférieure, et le temps moyen d’exécution par instance. Cette organisation permet de comparer FFD, HGGA et RL-HGGA dans des conditions homogènes, et d’analyser le compromis entre qualité de solution et temps de calcul. Une étude d’ablation est également réalisée en comparant HGGA sans ML avec RL-HGGA, afin d’évaluer l’apport réel du composant Q-learning.

TABLE I
ENVIRONNEMENT MATÉRIEL ET LOGICIEL DES EXPÉRIMENTATIONS.

Élément	Configuration
Machine	ASUS Vivobook X7600PC_N7
Système	Ubuntu 24.04.4 LTS x86_64, noyau 6.17.0-35-generic
CPU	Intel Core i7-11370H, 8 threads logiques
Mémoire	16 GiB RAM
Environnement d’exécution	Jupyter Notebook / Python
Bibliothèques principales	NumPy, Pandas, Matplotlib ; scikit-learn utilisé lorsque nécessaire

B. Jeux de données

Les expériences utilisent des benchmarks classiques du 1D-BPP, en particulier les familles Falkenauer (T et U), Scholl et Hard28. Ces familles couvrent des tailles d’instances variées et des structures de poids différentes, ce qui permet d’évaluer à la fois la qualité moyenne, la robustesse et la montée en coût des méthodes lorsque le nombre d’objets augmente [11].

TABLE II
FAMILLES DE BENCHMARKS UTILISÉES.

Famille	Nb. inst.	Taille n	Référence
Falkenauer T60	20	60	Qualité / petit format
Falkenauer T120	20	120	Qualité / format moyen
Falkenauer T249	20	249	Qualité / format moyen
Falkenauer T501	20	501	Grande taille
Falkenauer U120–U1000	80	120–1000	Robustesse
Scholl 1–3	variable	50–500	Diversité d’instances
Hard28	28	160–200	Instances difficiles

““latex

C. Calibrage des paramètres

HGGA et RL-HGGA sont évalués avec les mêmes paramètres principaux afin de garantir une comparaison équitable. Les valeurs retenues sont $\mu = 25$, $max_gen = 80$ et $patience = 25$. Ces paramètres contrôlent directement le compromis entre qualité de solution et temps de calcul.

Le paramètre μ désigne la taille de la population, c’est-à-dire le nombre de chromosomes maintenus à chaque génération. Dans notre cas, $\mu = 25$ signifie que l’algorithme conserve 25 solutions candidates en parallèle. Une population plus grande peut augmenter la diversité et réduire le risque de convergence prématurée, mais elle augmente aussi le temps

d’exécution, car davantage de solutions doivent être croisées, mutées et évaluées à chaque génération.

Le paramètre max_gen représente le nombre maximal de générations autorisées. Une génération correspond à une itération complète de la boucle évolutionnaire : sélection des parents, application des opérateurs, production de nouveaux individus, évaluation, puis remplacement. Avec $max_gen = 80$, l’algorithme peut donc effectuer au maximum 80 cycles d’amélioration pour une instance donnée. Ce paramètre joue le rôle d’un budget de recherche : une valeur plus élevée laisse plus de temps à l’algorithme pour améliorer les solutions, mais augmente le coût de calcul.

Le paramètre $patience$ est un critère d’arrêt anticipé basé sur la stagnation. Dans notre configuration, $patience = 25$ signifie que si la meilleure solution ne s’améliore pas pendant 25 générations consécutives, l’exécution s’arrête avant d’atteindre max_gen . Ce mécanisme évite de continuer inutilement la recherche lorsque la population semble bloquée autour d’une même qualité de solution. Il permet donc de réduire le temps d’exécution sans imposer un arrêt trop précoce.

Pour l’entraînement de RL-HGGA, l’agent Q-learning est entraîné sur 400 épisodes. Chaque épisode correspond à une exécution de la boucle RL-HGGA sur une instance d’entraînement. L’agent utilise une stratégie ϵ -greedy : au début, ϵ est élevé afin de favoriser l’exploration des différentes macro-actions ; ensuite, ϵ décroît progressivement jusqu’à 0.05 afin de privilégier les actions qui ont obtenu les meilleures valeurs dans la table $Q(s, a)$. Ce réglage permet à l’agent d’explorer plusieurs comportements au début de l’entraînement, puis d’exploiter progressivement la politique apprise. ““

D. Résultats principaux : qualité des solutions

La qualité des solutions est d’abord analysée à travers le gap moyen par rapport à la borne inférieure. La Fig. 2 compare FFD, HGGA et RL-HGGA sur les différentes familles d’instances. On observe que FFD présente les gaps les plus élevés, en particulier sur les familles Falkenauer T. Ce résultat confirme la limite des heuristiques gloutonnes : elles produisent très rapidement une solution faisable, mais ne disposent pas de mécanisme d’amélioration capable de réorganiser les bins après les premières décisions.

HGGA obtient généralement les meilleurs gaps moyens. Cela s’explique par sa capacité à explorer plusieurs regroupements possibles grâce au croisement BPCX, aux mutations et à la recherche locale. RL-HGGA reste nettement meilleur que FFD et se rapproche souvent de HGGA, mais il ne surpasse pas systématiquement HGGA en qualité pure. Ce point est important pour l’interprétation des résultats : l’apport du composant ML ne se traduit pas principalement par une amélioration absolue du nombre de bins, mais par une meilleure gestion du compromis entre qualité de solution et temps de calcul.

La Fig. 3 complète cette analyse en comparant le nombre moyen de bins utilisés par chaque méthode. Les résultats confirment la tendance observée avec le gap : FFD utilise généralement plus de bins, tandis que HGGA et RL-HGGA

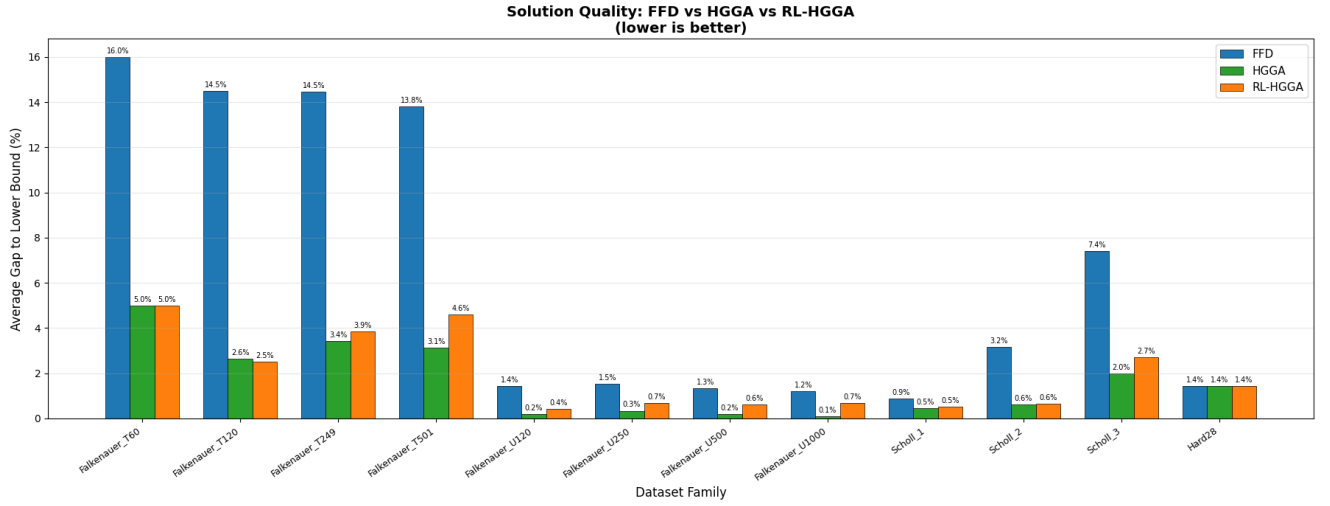


FIGURE 2. Gap moyen à la borne inférieure par famille : comparaison entre FFD, HGGA et RL-HGGA. Plus la valeur est faible, meilleure est la solution.

produisent des packings plus compacts. Sur plusieurs familles, RL-HGGA obtient un nombre de bins proche de celui de HGGA, ce qui montre que l'agent Q-learning conserve une bonne qualité de solution malgré une stratégie de recherche plus rapide.

E. Résultats principaux : temps d'exécution

La Fig. 4 présente les temps moyens d'exécution par famille, en échelle logarithmique. FFD est logiquement la méthode la plus rapide, car elle applique une construction gloutonne sans phase d'amélioration. HGGA est beaucoup plus coûteux, car il maintient une population de solutions et applique des opérateurs évolutionnaires sur plusieurs générations.

RL-HGGA se situe entre ces deux extrêmes, mais il est beaucoup plus proche de FFD que de HGGA en temps d'exécution. Cela montre que l'agent Q-learning apprend à privilégier des séquences d'opérateurs moins coûteuses, tout en conservant une qualité de solution acceptable. Le résultat principal de RL-HGGA est donc un compromis qualité-temps : la méthode n'améliore pas toujours HGGA en qualité pure, mais elle réduit fortement le temps de calcul par rapport à HGGA.

TABLE III
RÉSUMÉ GLOBAL DES RÉSULTATS PRINCIPAUX.

Méthode	Gap moyen	Temps moyen par instance
FFD	2.47%	0.0008 s
HGGA	0.75%	64.22 s
RL-HGGA	0.95%	1.29 s

Le tableau III synthétise ce compromis. HGGA obtient le meilleur gap moyen global, avec 0.75%, mais son temps moyen atteint 64.22 secondes par instance. RL-HGGA obtient un gap moyen légèrement supérieur, 0.95%, mais avec un temps moyen fortement réduit, égal à 1.29 seconde par instance. Ainsi, l'apport du Q-learning n'est pas de remplacer

HGGA en qualité maximale, mais de rendre la recherche plus efficace en temps.

F. Analyse du comportement de RL-HGGA

Pour comprendre le comportement de RL-HGGA, il faut analyser à la fois sa phase d'entraînement, sa politique apprise et son comportement de convergence.

La Fig. 5 montre l'évolution du gap pendant l'entraînement ainsi que la décroissance du taux d'exploration ϵ . Au début de l'entraînement, l'agent explore fortement les différentes actions disponibles. Progressivement, ϵ diminue, ce qui signifie que l'agent exploite davantage les actions ayant obtenu les meilleures valeurs dans la table $Q(s, a)$. Le gap d'entraînement reste variable, ce qui est normal car les épisodes portent sur des instances différentes, mais la courbe moyenne indique que l'agent apprend une politique exploitable.

La Fig. 6 présente la distribution des actions apprises par la politique gloutonne. On observe que l'agent ne sélectionne pas toutes les actions de manière uniforme. Il privilégie principalement certaines macro-actions, notamment les combinaisons basées sur BPCX avec mutation légère, ainsi que la mutation seule. Cela indique que l'agent a appris à éviter certaines actions plus coûteuses ou moins rentables dans le cadre de la récompense utilisée.

La Fig. 7 compare l'évolution de HGGA et RL-HGGA sur une instance de référence. RL-HGGA atteint rapidement une bonne qualité, ce qui confirme son orientation vers une recherche plus efficace en temps. HGGA peut conserver un léger avantage final en qualité sur certaines instances, mais au prix d'un temps d'exécution beaucoup plus élevé.

G. Étude par ablation : apport réel du composant ML

L'étude d'ablation consiste à comparer la métaheuristique HGGA sans composant ML avec RL-HGGA, qui conserve les mêmes opérateurs mais remplace la sélection fixe des opérateurs par un agent Q-learning. Cette comparaison permet d'isoler l'effet du module d'apprentissage.

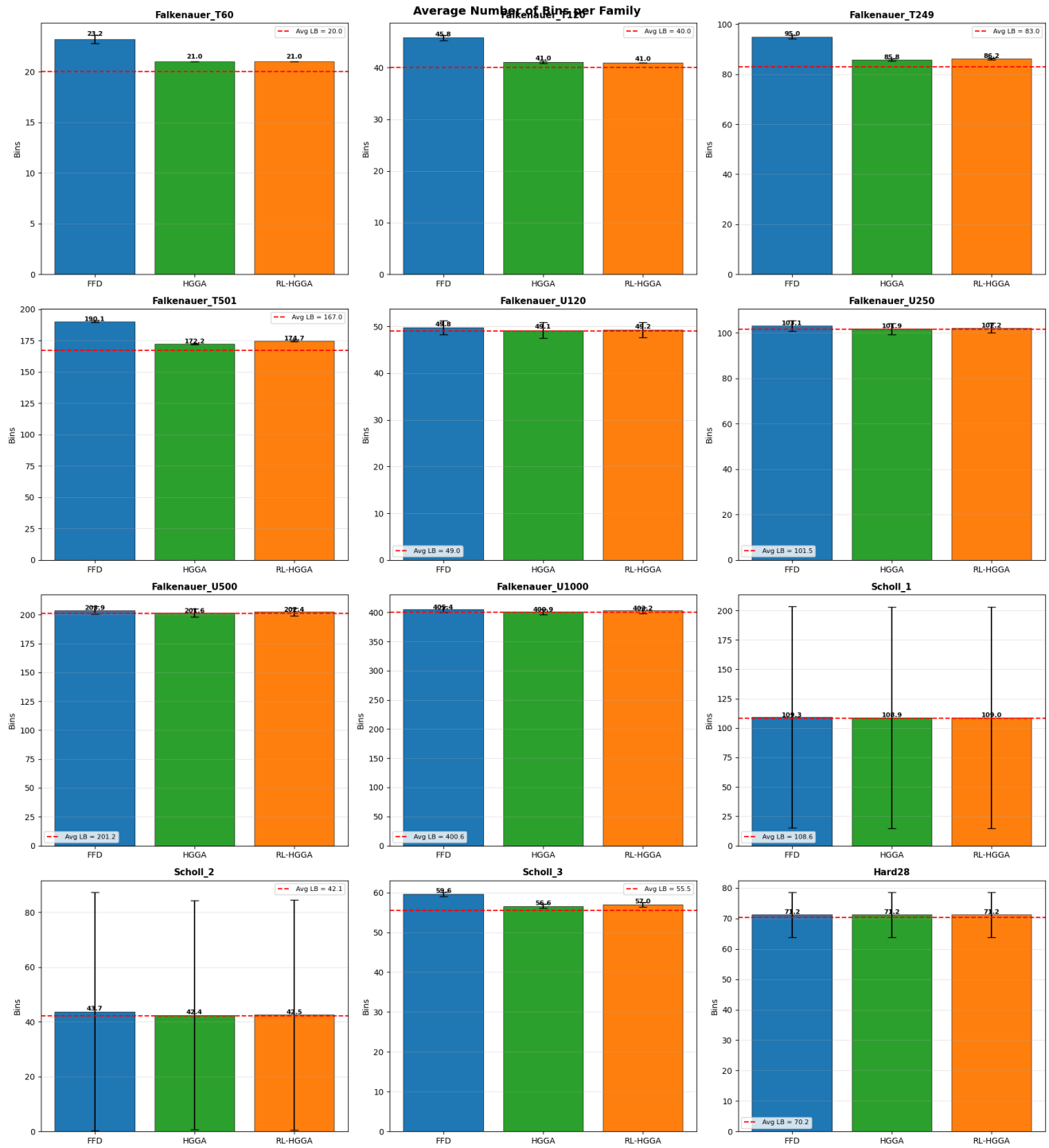


FIGURE 3. Nombre moyen de bins utilisés par famille : comparaison entre FFD, HGGA et RL-HGGA.

Les résultats montrent que l'ajout du Q-learning modifie principalement le comportement temporel de l'algorithme. HGGA obtient un meilleur gap moyen global, avec 0.75%, alors que RL-HGGA obtient 0.95%. En revanche, RL-HGGA réduit fortement le temps moyen d'exécution, passant de 64.22 secondes pour HGGA à 1.29 seconde par instance. L'apport du ML est donc réel, mais il doit être interprété correctement : il

ne consiste pas à améliorer systématiquement la qualité finale, mais à apprendre une politique de recherche plus rapide tout en gardant une qualité proche de celle de HGGA.

D'un point de vue critique, cette ablation montre que le composant ML apporte un gain important en efficacité, mais qu'il introduit aussi une légère perte de qualité moyenne. Le choix de la fonction de récompense est donc déterminant.

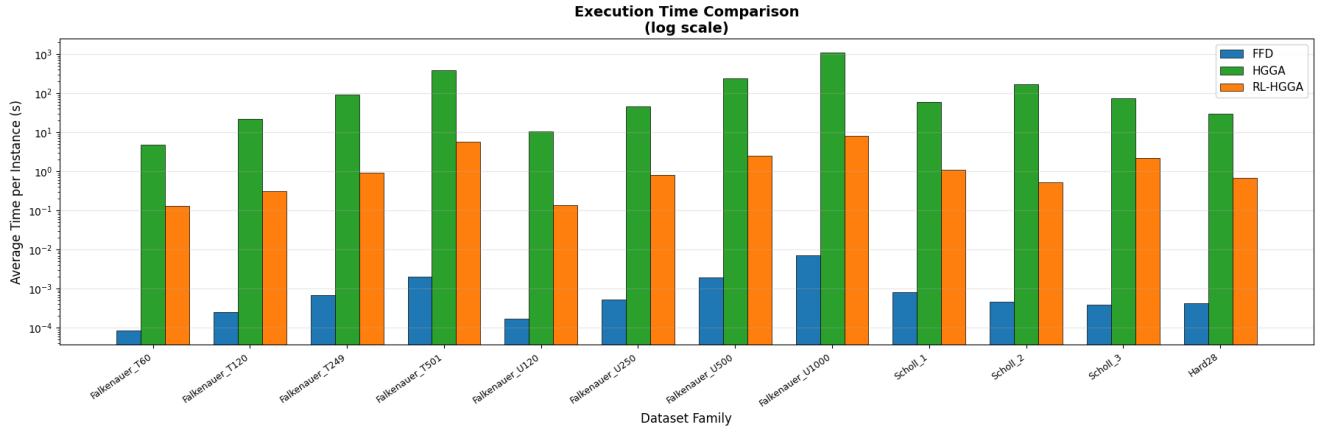


FIGURE 4. Comparaison des temps d'exécution moyens par famille entre FFD, HGGA et RL-HGGA. L'échelle logarithmique permet de visualiser les grands écarts de temps.

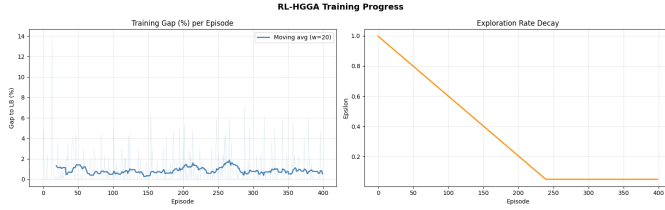


FIGURE 5. Progression de l'entraînement de RL-HGGA : évolution du gap d'entraînement et décroissance du taux d'exploration ϵ .

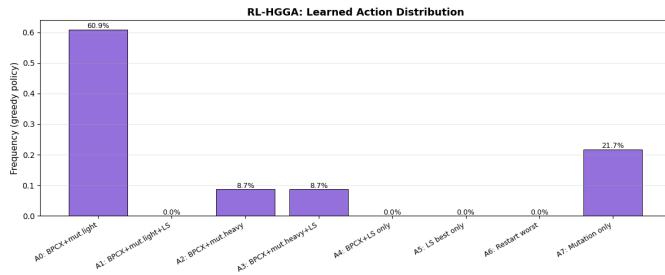


FIGURE 6. Distribution des macro-actions apprises par RL-HGGA sous la politique gloutonne.

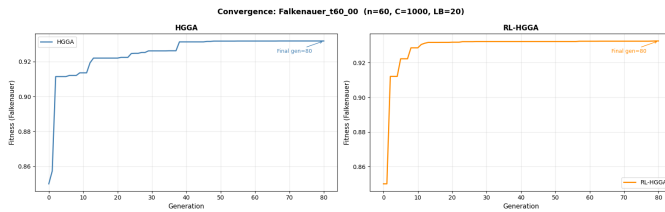


FIGURE 7. Comparaison du comportement de convergence entre HGGA et RL-HGGA sur une instance de référence.

Une récompense trop orientée vitesse favorise des actions peu coûteuses, tandis qu'une récompense trop orientée qualité risque d'augmenter le temps de calcul. Le bon comportement de RL-HGGA dépend donc de l'équilibre entre amélioration de la solution et coût temporel.

H. Étude comparative

L'étude comparative confronte quantitativement les méthodes implémentées dans ce projet : FFD, HGGA et RL-HGGA. FFD représente la famille des heuristiques constructives rapides. HGGA représente la métaheuristique de groupement utilisée comme référence sans composant intelligent. RL-HGGA constitue l'approche proposée, dans laquelle l'apprentissage par renforcement pilote dynamiquement le choix des opérateurs.

Les résultats montrent que FFD est la meilleure méthode en temps, mais la moins performante en qualité. HGGA obtient le meilleur gap moyen global, mais son temps d'exécution est beaucoup plus élevé. RL-HGGA se positionne comme une méthode intermédiaire : elle conserve une qualité nettement meilleure que FFD et proche de HGGA, tout en réduisant fortement le temps d'exécution par rapport à HGGA.

Concernant la comparaison avec la littérature, il est important de préciser que nous n'avons pas réalisé une confrontation numérique directe avec les valeurs publiées dans les articles existants. Une telle comparaison demanderait les mêmes instances exactes, les mêmes critères d'arrêt, le même environnement matériel et le même protocole statistique. Dans ce travail, la littérature est donc utilisée comme **référence méthodologique** et comme positionnement scientifique. En particulier, l'étude de Munien *et al.* confirme que les métaheuristiques appliquées au 1D-BPP présentent un compromis fort entre qualité de solution et temps de calcul [10]. Notre contribution s'inscrit dans cette continuité : nous comparons nos méthodes implémentées entre elles, puis nous montrons que l'ajout du Q-learning vise précisément à mieux contrôler ce compromis.

Ainsi, l'étude comparative doit être interprétée de manière prudente. Elle valide expérimentalement, sur notre protocole,

que RL-HGGA améliore l'efficacité temporelle par rapport à HGGA tout en conservant une qualité proche. Elle ne prétend pas démontrer que RL-HGGA surpasse quantitativement tous les résultats de la littérature, car cette comparaison n'a pas été menée dans des conditions strictement identiques.

I. Discussion

Au vu des résultats, trois conclusions ressortent clairement.

- **FFD** est indispensable comme baseline de rapidité, mais insuffisant lorsque la qualité de la solution est prioritaire.
- **HGGA** reste la référence en qualité moyenne ; c'est la meilleure baseline métaheuristique parmi les méthodes étudiées.
- **RL-HGGA** est la contribution principale du projet, car il apporte un compromis qualité-temps nettement plus intéressant que HGGA lorsque le coût de calcul doit être maîtrisé.

Autrement dit, le composant ML ne remplace pas l'expertise heuristique portée par HGGA ; il la **pilote**. C'est précisément cette orchestration adaptative qui constitue l'intérêt du travail.

V. CONCLUSION ET PERSPECTIVES

Ce travail a proposé une hybridation entre HGGA et l'apprentissage par renforcement pour le 1D-BPP. L'approche RL-HGGA conserve les opérateurs efficaces de la métaheuristique de référence et rend adaptatif leur mécanisme de sélection à l'aide d'un agent Q-learning. Les expériences montrent que HGGA reste la meilleure méthode en qualité moyenne, tandis que RL-HGGA fournit un compromis qualité-temps très intéressant, avec une réduction importante du temps d'exécution pour une perte de qualité limitée. L'étude d'ablation confirme en outre que le composant Q-learning apporte surtout un gain d'efficacité temporelle, avec une légère perte de qualité moyenne par rapport à HGGA.

Comme perspectives, plusieurs axes apparaissent naturellement : enrichir l'état RL par des indicateurs plus fins de diversité, raffiner l'espace d'actions, apprendre non seulement le type d'opérateur mais aussi son intensité, ou encore remplacer le Q-learning tabulaire par un apprentissage profond lorsque l'espace d'états devient trop riche. Une autre piste consisterait à intégrer des mécanismes de *hyper-heuristics* plus généraux pour sélectionner non seulement les opérateurs, mais aussi certains paramètres de l'algorithme en ligne.

RÉFÉRENCES

- [1] M. R. Garey and D. S. Johnson, *Computers and Intractability : A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [2] D. S. Johnson, A. Demers, J. D. Ullman, M. R. Garey, and R. L. Graham, "Worst-case performance bounds for simple one-dimensional packing algorithms," *SIAM Journal on Computing*, vol. 3, no. 4, pp. 299–325, 1974.
- [3] E. Falkenauer, "A hybrid grouping genetic algorithm for bin packing," *Journal of Heuristics*, vol. 2, pp. 5–30, 1996.
- [4] S. Martello and P. Toth, *Knapsack Problems : Algorithms and Computer Implementations*. Wiley, 1990.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning : An Introduction*, 2nd ed. MIT Press, 2018.
- [6] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [7] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, S. Ozcan, and R. Qu, "Hyper-heuristics : A survey of the state of the art," *European Journal of Operational Research*, vol. 229, no. 3, pp. 329–341, 2013.
- [8] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization : a methodological tour d'horizon," *European Journal of Operational Research*, vol. 290, no. 2, pp. 405–421, 2021.
- [9] E. Khalil, H. Dai, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," in *Advances in Neural Information Processing Systems*, 2017.
- [10] C. Munien, S. Mahabeer, E. Dzitiro, S. Singh, S. Zungu, and A. E.-S. Ezugwu, "Metaheuristic approaches for one-dimensional bin packing problem : A comparative performance study," *IEEE Access*, vol. 8, pp. 227438–227465, 2020, doi : 10.1109/ACCESS.2020.3046185.
- [11] M. Delorme, M. Iori, and S. Martello, "BPPLIB : a library for bin packing and cutting stock problems," *Optimization Letters*, vol. 12, pp. 235–250, 2018.